Agenda:

1. Bitwise Operations [AND, OR, XOR & NOT]
2. Basic AND XOR OR Properties
3. Negative Numbers
4. Ranges
5. Importance of Constraints
6. Left Shift & Right Shift

Bits: 0 => Off / Unset Bit / False

1 => On / Set Bit / True

Decimal of 10100 = $(20)_{10}$

Binary of 45 = $(101101)_2$

Bitwise Operations [AND, OR, XOR & NOT]

↗ Addition without carry

a	b	a&b (AND)	a|b (OR)	a^b (XOR)	~a (NOT)
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

↓

same → 0
different → 1

int → 32 bit

20 & 45 = ? a&b <= min(a, b)

	5	4	3	2	1	0
20 →	0	1	0	1	0	0
45 →	1	0	1	1	0	1
	0	0	0	1	0	0 → 4

20 | 45 = ? a | b >= max(a, b)

	5	4	3	2	1	0
20 →	0	1	0	1	0	0
45 →	1	0	1	1	0	1
	1	1	1	1	0	1

32 + 16 + 8 + 4 + 1 = 61

20 ^ 45 = ?

20 →	0	1	0	1	0	0
45 →	1	0	1	1	0	1
	1	1	1	0	0	1 → 57

Basic Properties

1. Even / Odd Number →

```
          3  2  1  0
11  →     1  0  1  1

&         0  0  0  1
---------------------
          0  0  0  1  → 1
---------------------
```

```
          3  2  1  0
10  →     1  0  1  0

&         0  0  0  1
---------------------
          0  0  0  0  → 0
---------------------
```

N & 1 → 1 ⇒ N is odd

→ 0 ⇒ N is even

A → any integer

A = 5 → 101

2. A & 0 → 0

```
  1 0 1
& 0 0 0
-------
0 0 0 → 0
```

3. A & A → A

```
  1 0 1
& 1 0 1
-------
1 0 1 → 5
```

4. **A | 0** → A

```
  1 0 1
| 0 0 0
-------
  1 0 1 → 5
```

5. **A | A** → A

```
  1 0 1
| 1 0 1
-------
  1 0 1 → 5
```

6. **A ^ 0** → A

```
  1 0 1
^ 0 0 0
-------
  1 0 1 → 5
```

7. **A ^ A** → 0

```
  1 0 1
^ 1 0 1
-------
  0 0 0 → 0
```

Cumulative Property →

```
A & B = B & A // Bitwise AND
A | B = B | A // Bitwise OR
A ^ B = B ^ A // Bitwise XOR
```

Associative Property →

```
(A & B) & C = A & (B & C) // Bitwise AND
(A | B) | C = A | (B | C) // Bitwise OR
(A ^ B) ^ C = A ^ (B ^ C) // Bitwise XOR
```

$2 \wedge 3 \wedge 4 \wedge 2 \wedge 4 = \underline{2 \wedge 2} \wedge \underline{4 \wedge 4} \wedge 3$

$= \underline{0 \wedge 0} \wedge 3 = 0 \wedge 3 = \underline{\underline{3}}$

Binary Representation of Negative numbers

To represent a negative number in binary representation, we use two's complement.

It works as follows -

- Convert the absolute value of number to binary representation.
- Invert all the bits of number obtained in step 1.
- Add 1 to the number obtained in step 2.

8 bit system

	7	6	5	4	3	2	1	0
45 →	0	0	1	0	1	1	0	1
1's comp. →	1	1	0	1	0	0	1	0
Add 1 →	+ 0	0	0	0	0	0	0	1
2's Comp. →	1	1	0	1	0	0	1	1

→ -45

$(-2^7) + 2^6 + 2^4 + 2^1 + 2^0$

$-128 + 64 + 16 + 2 + 1 = -128 + 83 = -45$

8 bit System

	7	6	5	4	3	2	1	0
MAX →	0	1	1	1	1	1	1	1

$2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 2^7 - 1 = 127$

MIN →	1	0	0	0	0	0	0	0

$-2^7 = -128$

$[-128 \quad 127]$

Range of Data types

8 bit system → $[-2^7 \quad 2^7-1]$ → $[-128 \quad 127]$

4 bit system → $[-2^3 \quad 2^3-1]$ → $[-8 \quad 7]$

$\begin{array}{lcccc} & 3 & 2 & 1 & 0 \\ \text{MAX} \rightarrow & 0 & 1 & 1 & 1 \end{array} \rightarrow 7$

MIN → 1 0 0 0 → -8

32 bit system → $[-2^{31} \quad 2^{31}-1]$ ← Integer

$\begin{array}{lcccccc} & 31 & 30 & \ldots & & & 0 \\ \text{MAX} \rightarrow & 0 & 1 & 1 & 1 & \ldots & 1 \end{array} \rightarrow 2^{30}+2^{29}+\ldots 2^0 = \underline{2^{31}-1}$

MIN → 1 0 0 . . . 0 → $\underline{-2^{31}}$

64 bit system → $[-2^{63} \quad 2^{63}-1]$ ← Long

$\begin{array}{lcccccc} & 63 & 62 & 61 & \ldots & & 0 \\ \text{MAX} \rightarrow & 0 & 1 & 1 & \ldots & & 1 \end{array} \rightarrow 2^{62}+2^{61}+\ldots 2^0 = \underline{2^{63}-1}$

MIN → 1 0 0 . . . 0 → $\underline{-2^{63}}$

---

8 bit system →

$$\begin{array}{lcccccccccc} & & {}^{1}7 & {}^{1}6 & 5 & 4 & 3 & 2 & 1 & 0 & \\ & & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 1 & \rightarrow 109 \\ + & & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & \rightarrow \underline{+48} \\ \hline & & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & \underline{157} \end{array}$$

$-2^7 + 2^4+2^3+2^2+2^0 = -128+29$

$= \underline{-99}$

Importance of Constraints

int a [10^9] → ~4GB

long a [10^9] → ~8GB

Q1 → Find product of two numbers A & B.

$1 <= A, B <= 10^5$

$A * B \rightarrow max \rightarrow 10^5 * 10^5 = 10^{10}$

int overflow

```
int c = A * B
print (c)
```

✗

```
long c = long (A * B)
print (c)
```

✗

```
long c = A * B
print (c)
```

✗

int overflow

int * int → int

```
long c = A * long(B)
print (c)
```

✓

int * long → long

Q2 → Find sum of array elements. $1 <= A[i] <= 10^6$

$|A| <= 10^5$

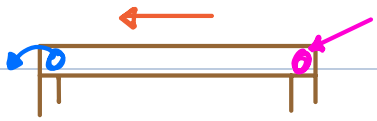
```
~~int~~ long sum = 0
for i → 0 to (N-1) {
    sum += A[i]
} return sum
```

max → $10^6 + 10^6 + 10^6 \ldots$ (10^5 times)

$= 10^6 * 10^5 = 10^{11}$

int overflow

Left Shift Operator (<<)



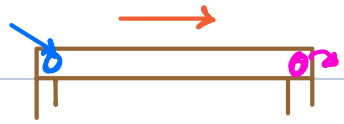
discarded

	7	6	5	4	3	2	1	0	
a =	0	0	0	0	1	0	1	0	→ 10
									*2
a << 1	0	0	0	1	0	1	0	0	→ 20
									*2
a << 2	0	0	1	0	1	0	0	0	→ 40
									*2
a << 3	0	1	0	1	0	0	0	0	→ 80
									*2
a << 4	1	0	1	0	0	0	0	0	→ 160
									*2
a << 5	0	1	0	0	0	0	0	0	~~320~~ → 64

$$\boxed{a << n = a * 2^n}$$

$$\boxed{1 << n = 2^n}$$

Right Shift Operator (>>)



	7	6	5	4	3	2	1	0 (discarded)	
a =	0	0	0	0	1	0	1	0	→ 10
a >> 1	0	0	0	0	0	1	0	1	→ 5
a >> 2	0	0	0	0	0	0	1	0	→ 2
a >> 3	0	0	0	0	0	0	0	1	1
a >> 4	0	0	0	0	0	0	0	0	0

$\downarrow /2$

$$\boxed{a \gg n = a/2^n}$$

$a = 12 \rightarrow$ 1 1 0 0

$b = 5 \rightarrow$ & 0 1 0 1

0 1 0 0

$c = 3 \rightarrow$ 1 0 0 1 1

0 1 1 1 → 7

$x = 1 << 10 \rightarrow 2^{10} = 1024$

$y = x << 1 \rightarrow 2^{11} = \underline{2048}$ ✓